

한충희

모바일 RTS / MMORPG 클라이언트 프로그래머 · 신입

010-2783-9414 | rislgowjra@gmail.com | 인천광역시 남동구

포트폴리오 huihans.github.io/Wooduduk.io



보유 기술

주력 Strong

C#
주 언어

Unity 6.3
URP · 게임 클라이언트

Firebase 실시간 멀티
서버리스 세션·동기화

결정론 설계
시드 기반 · 재현 가능

사용 경험 Used

PlayFab
써드파티 SDK 연동

Apps Script 파이프라인
Sheets 파싱·검증

EditMode 테스트
단위 검증

협업·도구 Tools

UVCS
브랜치·머지 협업

WBS
작업 분해·일정

Git
버전 관리

설계 강점

이벤트버스 모듈화

인터페이스 경계 설계

ScriptableObject 데이터 구동

프로젝트

우드득!(Wooduduk) — 실시간 멀티 스코어어택 게임

기간 2026.06.03~07.02

팀 6명 · 클라이언트 시스템 파트

플랫폼 PC · 모바일

엔진 Unity 6.3 (URP) / C#

슬롯을 굴려 나온 결과로 장작을 패고, 최대 4인이 실시간으로 점수·생존을 겨루는 캐주얼 대전 게임.

본인 담당

실시간 멀티플레이 (최대 4인 · Firebase)

매칭 시스템

데이터 기반 카드·효과 시스템

기획자용 데이터 파이프라인

AI 빌드 전 코드 점검

규모 · 우드득 전체 라이브 코드 **183개** 중 실시간 멀티·티어·카드 시스템 담당 · 트러블슈팅 **4건** 근본 원인까지 해결

포트폴리오 huihans.github.io/Wooduduk.io

이전 프로젝트

LuckyRandomDefense · 2D 랜덤 디펜스 · 팀 · 2026.04~05 — 팀장·PM · PlayFab 클라우드 세이브·데이터(백엔드) · 레벨 디자인

Scene Thriller · 1인칭 사운드 퍼즐 공포 방탈출 · 개인 · 2026.03 — 리소스 직접 수집 · 방 텍스처·게임 기능 구현

교육 · 기타

• 넥스트러너스 게임개발 부트캠프 · 2025.12~2026.07 · 수료

• 병역 · 만기전역

• 운산공업고등학교 졸업 · 2018.02

자기소개서

한충희 · (주)이스트게임즈 클라이언트 프로그래머(신입)

오래 손보며 굴러가는 게임을 만들고 싶습니다 — 지원 동기

제가 만든 게임들은 한 번 하고 끝나는 게 아니라, 데이터를 계속 고치고 콘텐츠를 더하며 굴러가는 구조였습니다. 서버에서 밸런스를 바꾸면 클라이언트 빌드 없이 반영되게 만들어 보면서, 오래 운영되는 게임의 재미는 '계속 손볼 수 있게 만들어 둔 구조'에서 나온다고 느꼈습니다.

그래서 MM팀이 새로 시작하는 미드코어 실시간 전략 대전에 마음이 갔습니다. 실시간으로 여러 유저의 상태를 어긋나지 않게 맞추고 공정성을 지키는 일은, 제가 우드득에서 씨름했던 문제와 그대로 맞닿아 있습니다. 자리 잡은 프로젝트에 합류하기보다 초반부터 함께 구조를 잡아 가며, 오래 굴러갈 게임을 밑바닥부터 만들어 보고 싶어 지원했습니다. '게임의 본질은 재미'라는 팀의 방향에도 깊이 공감합니다.

문제를 증상이 아니라 원인에서 끝냅니다 — 문제 해결

우드득 막바지에, 특정 계정만 멀티가 시작되지 않고 멈추는 버그가 있었습니다. 재현이 불규칙해 원인을 바로 알 수 없어 로그를 붙여 추적했습니다. 호스트를 '접속자 중 가장 작은 uid'로 뽑는 로직이 있었는데, 정원을 채우려 넣은 AI 대역(고스트)이 그 후보에 섞여 있었습니다. 실제 유저의 uid가 사전순으로 고스트보다 뒤면 고스트가 호스트로 뽑혔고, 고스트는 시작 시각을 기록하지 않아 나머지가 영원히 대기했습니다.

눈에 보이는 증상(무한 대기)을 우회하는 대신, 호스트 선출에서 고스트를 아예 제외하고 '실제 유저가 한 명도 없는 방은 파기한다'는 안전망까지 넣어 원인을 없앴습니다. uid 순서에 따라 됐다 안 됐다 하던 현상이 사라졌습니다. 재현이 어려운 버그일수록 증상이 아니라 원인을 찾아 끝내는 것이, 여러 사람이 오래 손대는 서비스에서 재발을 막는다고 배웠습니다.

혼자에서 팀으로, 기능에서 시스템으로 — 성장 경험

세 프로젝트를 거치며 다루는 범위가 넓어졌습니다. 첫 개인 프로젝트(Scene Thriller)로 게임 하나를 혼자 끝까지 완성해 봤고, 첫 팀 프로젝트(LuckyRandomDefense)에서는 팀장을 맡으며 PlayFab으로 익명 로그인-클라우드 세이브-서버 밸런스 데이터 로딩을 붙이는 '서버 데이터' 파트를 담당했습니다. 우드득에서는 그 데이터 경험을 Firebase 실시간 동기화로 이어, 전용 서버 없이 최대 4인 실시간 대전을 만들었습니다.

프로젝트가 커질수록 '나 하나 잘 짜는 것'보다 '남이 기다리지 않게, 나중에 고치기 쉽게' 구조를 갖는 일이 중요해졌습니다. 백엔드가 없던 때 외부 의존을 인터페이스로만 받아 제 모듈을 따로 검증하고, 슬롯 이벤트를 정적 이벤트버스로 떼어 담당자끼리 파일이 엉키지 않게 한 것도 그래서였습니다. 세 프로젝트의 코드 의존 구조를 직접 그래프로 정리해 포트폴리오에 담기도 했습니다. (AI 도구는 빌드 전 코드 점검처럼 반복 작업을 줄이는 데 쓰되, 판단과 구조는 제가 잡았습니다.)

실시간-데이터 경험을 RTS에서 이어가겠습니다 — 입사 후

여러 클라이언트의 상태를 어긋나지 않게 맞추고 같은 입력이면 같은 결과가 나오도록 결정론적으로 설계한 경험은, 유닛과 자원이 실시간으로 얹히는 RTS에서도 통하는 부분이 있다고 생각합니다. 이스트게임즈의 코드베이스와 운영 방식은 입사 후 익혀야 하지만, 재현이 어려운 문제를 원인까지 추적하고 나중에 고치기 쉬운 구조로 남기는 일에는 직접 보탬이 될 수 있다고 생각합니다. 선배들의 손이 덜 가도록, 검증할 수 있는 결과물과 분명한 기록으로 일하겠습니다.